

이지패스 고래책 이벤트

급한 사람을 위한 SQLD 핵심정리노트



최신
기술 내용 반영

핵심문제/기본문제
모의고사/기출문제



기출 및 모의고사 수록
수험서 앱 제공

데이터
자격증 시리즈
EASY
PASS!



홈페이지 <https://wikibook.co.kr/sqld2026/>

예제파일 <https://github.com/wikibook/sqld>

SQL 초보자를 위해
친절하게 설명한

2026 SQLD SQL 개발자

전용문 지음, 송영민 감수



총 9회차 모의고사,
기출문제 제공(수험용 앱)



온라인 스터디 저자 직답
Q&A 커뮤니티 운영



핵심 요약 기출문제풀이
(유튜브 무료 강의)

◆ 한달 만에 끝내는 SQLD ◆

- 시간이 부족한 수험생을 위해 빠르고 효율적인 공부를 돕는 구성
- 초심자, 비전공자도 쉽게 이해할 수 있도록 구성
- 최신 출제 경향을 반영한 핵심 이론과 예상문제
- 유튜브 무료 강의(핵심요약, 기출문제풀이) 제공
- SQL 실습 코드 수록
- 온라인 스터디 및 Q&A 커뮤니티 운영

SQLD 수험생들을 위한 Q&A 커뮤니티 운영

- 저자가 직접 답변하는 Q&A 게시판
- 무료 온라인 스터디 운영
- 수험생들끼리 실시간 질문 답변이 가능한 SQLD 오픈카톡방 운영

▶
Q&A
커뮤니티



{EASY PASS}

2026 ADSP 데이터분석 준전문가 SQL 개발자



카테고리 | 나의활동

{EASY PASS}

위키박스 [대니얼]
2021.02.03. 개설
카테고리

열매1단계

25,931 글

초대

카테고리 글쓰기

카테고리 채팅

검색

로그인 | 회원가입

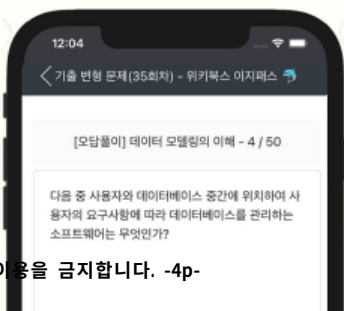
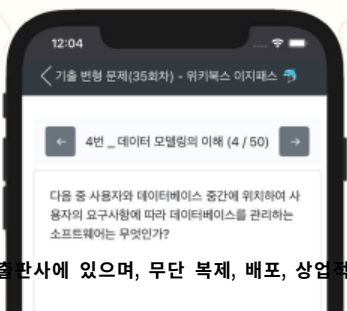
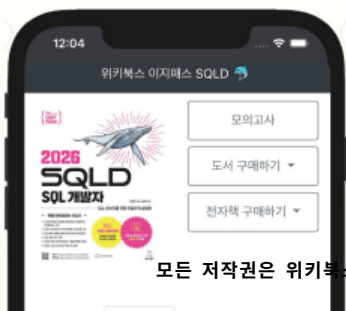
시험 일정

구분	2026년		접수기간	시험일	사전접수 발표	결과 발표
데이터분석 준전문가 (ADSP)	제48회	필기	1.5~1.9	2.7(토)	2.27~3.3	3.6
	제49회	필기	4.13~4.17	5.17(일)	5.29~6.2	6.5
	제50회	필기	7.6~7.10	8.8(토)	8.21~8.25	8.28
	제51회	필기	9.28~10.2	10.31(토)	11.13~11.17	11.20
SQL 개발자 (SQLD)	제60회	필기	2.2~2.6	3.7(토)	3.20~3.24	3.27
	제61회	필기	4.27~5.1	5.31(일)	6.12~6.16	6.19
	제62회	필기	7.20~7.24	8.22(토)	9.4~9.8	9.11

자투리시간 활용을 위한 수험용 앱 활용

- 총 9회차의 기출 문제, 모의고사 수록
- 자투리 시간을 활용해 앱으로 틈틈이 학습 가능

▶
수험용 앱



암기 노트



1과목 데이터 모델링의 이해 - 1장 데이터 모델링의 이해

학습목표

- 데이터 모델링의 개념, 중요성, 단계, 구성 요소 및 표기법을 이해하기
- 관계형 데이터베이스의 개념, 특징, SQL의 역할과 종류, 테이블 구조, 데이터 타입 등 파악하기
- 엔터티, 속성, 관계, 식별자의 개념과 특성을 정확히 이해하고 활용할 수 있기

목차

1. 데이터 모델의 이해
 2. 관계형 데이터베이스 개요
 3. 엔터티
 4. 속성
 5. 관계
 6. 식별자
-

1. 데이터 모델의 이해

1.1 모델링의 정의

- **모델링(Modeling)**: 현실 세계의 복잡한 현상이나 사물을 단순화하여 표현하는 기법
- **데이터진흥원 정의**: 사람이 접하는 다양한 현상(사람, 사물, 개념 등)을 약속된 방식(수식, 그림 등)으로 표현하는 것

1.2 모델링의 종류

- **정보시스템 모델링**: 정보 모델링, 데이터 모델링, 프로세스 모델링 등
- **수리 모델링**: 수학적 공식으로 현상을 표현
- **확률/통계 모델링**: 통계적 방법론으로 미래를 예측
- **회로 모델링**: 전자회로를 설계하고 구성

1.3 모델링의 특징

- **추상화**: 현실 세계를 일정한 형식에 맞추어 간략하게 표현

- **단순화:** 복잡한 현실을 기호, 문자, 그림 등으로 쉽게 이해할 수 있게 함
- **명확화:** 애매모호함을 제거하고 명확하게 현상을 기술

1.4 정보시스템 모델링의 세 가지 관점

- **데이터 관점 (What, Data):** 업무가 어떤 데이터와 관련되는지 모델링
- **프로세스 관점 (How, Process):** 업무가 실제로 처리하는 일을 모델링
- **데이터와 프로세스의 상관 관점:** 업무 처리 방식에 따라 데이터가 어떤 영향을 주고받는지 모델링

1.5 데이터 모델링의 정의

- **데이터 모델링:** 정보시스템 구축을 위해 데이터 관점에서 업무를 분석하고 현실 세계의 데이터를 약속된 표기법으로 표현하는 과정
- **데이터 모델이 제공하는 기능:**
 - 시스템 가시화 지원
 - 시스템 구조와 행동 명세화
 - 구조화된 틀 제공
 - 문서화
 - 다양한 관점 제공
 - 상세 수준의 구체화

1.6 데이터 모델링의 중요성 및 유의점

- **데이터 설계의 중요성:**
 - 시스템 구축에서 매우 중요한 요소
 - 복잡한 정보 요구사항을 간결하게 표현
 - 데이터 중복/비유연성/비일관성을 줄여 품질 향상
- **데이터 모델링 시 유의점:**
 - 중복된 데이터 요소 최소화
 - 변화 가능성 고려 (업무 변화에 유연하게 대응)
 - 데이터 모델의 단순화 추구
 - 데이터 재사용성 고려
 - 일관성과 통합성 유지

1.7 데이터 모델링의 3단계

- **개념적 데이터 모델링:**
 - 추상화 수준이 높고 업무 중심적, 포괄적 수준의 모델링
 - 전사적 관점에서 기업의 데이터 모델링 진행

- 주로 ERD 산출물 생성
- **논리적 데이터 모델링:**
 - 시스템 구축을 위한 키, 속성, 관계 등을 정확하게 표현
 - 재사용성이 높은 설계가 가능
 - 정규화를 통해 데이터 모델 품질 향상
- **물리적 데이터 모델링:**
 - 실제 DB 구축을 위한 설계 단계
 - 성능, 저장 등을 고려한 실제적 구현 방법 제시
 - 인덱스, 역정규화 등의 기법 사용

1.8 데이터 독립성 (Three-Schema Architecture)

- **외부 스키마 (External Schema):**
 - 사용자 관점의 스키마
 - 개인적인 데이터베이스 뷰
 - 응용 프로그램이 접근하는 영역
- **개념 스키마 (Conceptual Schema):**
 - 통합 관점의 스키마
 - 조직 전체의 데이터베이스를 기술
 - 전체 데이터베이스의 구조 정의
- **내부 스키마 (Internal Schema):**
 - 물리적 저장구조
 - 실제 데이터가 저장되는 방법 정의
- **독립성 종류:**
 - **논리적 독립성:** 개념 스키마가 변경되어도 외부 스키마에 영향 없음
 - **물리적 독립성:** 내부 스키마가 변경되어도 개념 스키마에 영향 없음
- **매핑(사상):** 상위 단계와 하위 단계 사이의 대응 관계

1.9 데이터 모델링의 세 가지 핵심 요소

- **엔터티 (Entity):** 업무에 필요하고 유용한 정보를 저장하는 집합적 실체
- **속성 (Attribute):** 엔터티를 구성하는 최소 데이터 단위
- **관계 (Relationship):** 엔터티 간의 논리적인 연관성

1.10 ERD (Entity-Relationship Diagram)

- **ERD 정의:** 엔터티와 엔터티 간의 관계를 그림으로 표현한 것
- **ERD 작성 순서:**

1. 엔터티 도출
2. 엔터티 간 관계 정의
3. 관계명 기술
4. 관계의 참여도 기술
5. 관계 필수여부 기술

- **ERD 표기법 종류:**

- **Chen 표기법:** 가장 오래된 표기법, 관계를 마름모꼴로 표시
- **IE (Information Engineering) 표기법:** 까마귀발 표기법이라고도 함, 관계를 선으로 표시
- **Barker 표기법:** 엔터티에 주식별자를 표기하지 않음, 관계는 선으로 표시

1.11 좋은 데이터 모델링의 특징

- **완전성:** 업무에 필요한 모든 데이터를 담아야 함
- **중복 배제:** 데이터 중복을 최소화하여 일관성 유지
- **업무 규칙 반영:** 현업의 업무 규칙을 정확히 표현
- **데이터 재사용:** 데이터 재사용이 용이하도록 설계
- **의사소통:** 비즈니스 요구사항을 명확히 전달
- **통합성:** 전체 데이터의 통합된 시각을 제공

2. 관계형 데이터베이스 개요

2.1 데이터베이스의 정의

- 데이터의 집합으로 조직적으로 구성된 데이터들의 모임
- **여러 관점의 정의:**
 - **법적 정의:** 저작물의 소재를 체계적으로 배열 또는 구성한 편집물
 - **기술적 정의:** 관련된 데이터의 모음으로, 중복을 통제하는 체계
 - **산업적 정의:** 조직체의 정보 요구를 지원하는 공유 데이터 자원

2.2 데이터베이스의 발전 과정

- **1960년대:** 파일 시스템 (파일 간 데이터 중복, 일관성/무결성 보장 어려움)
- **1970년대:** 계층형/망형 DBMS (자료 간 관계가 복잡, 유연성 부족)
- **1980년대:** 관계형 DBMS 등장 (오라클, DB2 등)
- **1990년대:** 객체 관계형 DBMS로 발전

2.3 관계형 데이터베이스의 특징

- 데이터 중복 최소화: 정규화를 통한 중복 제거
- 동시성 관리: 다중 사용자 환경에서 데이터 일관성 보장
- 병행제어: 동시에 여러 사용자가 데이터 처리할 때 일관성 유지
- 메타데이터 관리: 데이터에 관한 데이터(데이터 사전) 관리
- 보안/무결성/장애복구 기능: 데이터 보호와 안정성 보장
- SQL 사용: 표준 질의어를 통한 데이터 정의/조작/제어

2.4 SQL (Structured Query Language)

- SQL의 정의: 관계형 데이터베이스에서 사용하는 표준 질의어
- SQL 표준화: ANSI SQL을 통한 표준화
- SQL 명령어 유형:
 - DML (Data Manipulation Language): SELECT, INSERT, UPDATE, DELETE
 - DDL (Data Definition Language): CREATE, ALTER, DROP, TRUNCATE
 - DCL (Data Control Language): GRANT, REVOKE
 - TCL (Transaction Control Language): COMMIT, ROLLBACK, SAVEPOINT

2.5 테이블 구조와 데이터 타입

- 테이블 구성 요소:
 - 컬럼 (Column): 속성, 필드
 - 행 (Row): 레코드, 튜플
 - 필드 (Field): 컬럼과 행이 만나는 지점, 실제 데이터가 저장됨
- 주요 데이터 타입:
 - 숫자형: INTEGER, NUMBER, DECIMAL 등
 - 문자형: CHAR, VARCHAR, TEXT 등
 - 날짜형: DATE, TIMESTAMP
 - 이진형: BLOB, BINARY
 - 불리언형: BOOLEAN
- CHAR vs VARCHAR: CHAR은 고정길이, VARCHAR는 가변길이 문자열 저장

3. 엔터티

3.1 엔터티의 정의

- **엔터티**: 업무에 필요하고 유용한 정보를 저장하고 관리하기 위한 집합적인 실체 또는 객체
- 데이터베이스에서는 테이블로 구현됨

3.2 엔터티의 특징

- **업무적 필요성**: 반드시 업무에서 필요하고 관리할 가치가 있어야 함
- **식별자**: 유일한 식별자(예: 학번, 사번)로 식별 가능해야 함
- **인스턴스 집합**: 2개 이상의 인스턴스(행)를 포함해야 함
- **업무 프로세스 참여**: 실제 업무 프로세스에 활용되어야 함
- **속성 포함**: 반드시 속성을 가져야 함
- **관계 존재**: 다른 엔터티와 최소 1개 이상의 관계가 필요

3.3 엔터티의 분류

- **유무형에 따른 분류**:
 - **유형 엔터티**: 물리적 형태가 있음 (예: 학생, 사원, 강사)
 - **개념 엔터티**: 개념적인 정보 (예: 조직, 보험상품)
 - **사건 엔터티**: 업무 수행에 따라 발생 (예: 주문, 청구)
- **발생시점에 따른 분류**:
 - **기본 엔터티**: 독립적으로 생성 가능 (예: 고객, 상품)
 - **중심 엔터티**: 기본 엔터티로부터 생성 (예: 계약, 주문)
 - **행위 엔터티**: 두 엔터티 간의 행위로 생성 (예: 주문목록, 사원변경이력)

3.4 엔터티 명명 규칙

- 현업업무에서 사용하는 용어 사용
- 약어 사용 지양
- 단수 명사 사용
- 모든 엔터티에서 유일한 이름 부여
- 생성 의미가 부여된 이름 사용
- **명명 방식**:
 - **CamelCase**: 각 단어의 첫 글자를 대문자로 표기 (예: CustomerOrder)
 - **SnakeCase**: 단어 사이에 언더바(_) 사용 (예: customer_order)

4. 속성

4.1 속성의 정의

- **속성:** 업무에서 필요로 하는 최소한의 데이터 단위로, 더 이상 분리되지 않는 정보
- 엔터티를 설명하는 데이터 요소

4.2 속성의 특성

- **업무 필요성:** 해당 업무에서 필요하고 관리해야 할 정보여야 함
- **함수적 종속성:** 정규화 이론에 따라 주식별자에 함수적 종속성을 가져야 함
- **단일값 원칙:** 하나의 속성은 한 개의 값만을 가짐
- **도메인:** 속성이 가질 수 있는 값의 범위를 정의

4.3 속성의 분류

- **특성에 따른 분류:**
 - **기본 속성:** 업무 분석 단계에서 바로 정의되는 속성 (예: 상품명)
 - **설계 속성:** 업무 규칙화, DB 설계 과정에서 도출되는 속성 (예: 예금분류코드)
 - **파생 속성:** 다른 속성에서 계산이나 변형으로 생성되는 속성 (예: 이자)
- **엔터티 내 역할에 따른 분류:**
 - **PK(Primary Key) 속성:** 엔터티를 식별하는 속성
 - **FK(Foreign Key) 속성:** 다른 엔터티와의 관계에서 포함된 속성
 - **일반 속성:** PK나 FK가 아닌 일반적인 속성

4.4 속성의 명명 규칙

- 현업에서 사용하는 이름 사용
- 약어 사용 지양
- 전체 시스템에서 유일한 이름 사용
- 명사형 사용
- 이름만으로도 의미 파악 가능하게

5. 관계

5.1 관계의 정의

- **관계:** 엔터티의 인스턴스 간에 논리적인 연관성을 가지는 형태나 행위
- 엔터티 간의 업무적 연관성을 표현

5.2 관계의 표현

- 2개의 엔터티 사이의 관계는 2개의 관계명을 가짐
 - 예: 고객은 주문을 생성한다 / 주문은 고객에 의해 생성된다

5.3 관계의 분류

- **종류:**
 - **존재에 의한 관계:** 항상 존재하는 관계 (예: 사원은 부서에 속한다)
 - **행위에 의한 관계:** 특정 행위로 발생하는 관계 (예: 고객이 상품을 구매)
- **카디널리티(참여자 수):**
 - **1:1 관계:** 양쪽 엔터티의 인스턴스가 1:1로 대응
 - **1:M 관계:** 한쪽 엔터티 인스턴스가 여러 인스턴스와 관계
 - **N:M 관계:** 양쪽 엔터티의 인스턴스가 다대다 관계

5.4 관계의 선택성

- **필수적 관계:** 반드시 관계를 가져야 함 (실선으로 표시)
- **선택적 관계:** 관계를 가질 수도 있고 아닐 수도 있음 (점선으로 표시)

5.5 관계명 표기 규칙

- 관계의 이름은 현재형 동사 사용
 - 관계 참여자의 관점에서 능동적으로 명명
 - 관계는 양방향으로 명명 (능동/수동)
-

6. 식별자

6.1 식별자의 정의

- **식별자:** 엔터티 내에서 인스턴스를 구별하는 속성 또는 속성 조합
- 논리 모델링에서는 '식별자', 물리 모델링에서는 '키(Key)'라는 용어 사용

6.2 식별자의 특성

- **유일성:** 인스턴스를 유일하게 구분할 수 있어야 함
- **최소성:** 필요 최소한의 속성으로 구성
- **불변성:** 값이 변하지 않아야 함
- **존재성:** 반드시 값이 존재해야 함 (NULL 허용 안됨)

6.3 식별자의 분류

- **대표성에 따른 분류:**
 - **주식별자:** 엔터티를 대표하는 식별자 (예: 사원번호)
 - **보조식별자:** 대체 식별 가능한 식별자 (예: 주민등록번호)
- **생성 방식에 따른 분류:**
 - **내부식별자:** 엔터티 내에서 스스로 생성되는 식별자 (예: 고객센터)
 - **외부식별자:** 다른 엔터티로부터 받아오는 식별자 (예: 주문의 고객센터)
- **속성의 수에 따른 분류:**
 - **단일식별자:** 하나의 속성으로 구성된 식별자 (예: 고객센터)
 - **복합식별자:** 여러 속성의 조합으로 구성된 식별자 (예: 주문번호+상세순번)
- **대체 가능성에 따른 분류:**
 - **본질식별자:** 업무적으로 자연 발생한 식별자 (예: 주민등록번호)
 - **인조식별자:** 시스템에서 임의로 부여한 식별자 (예: 일련번호)

6.4 식별자 도출 기준

- 업무에서 자주 사용되는 속성 우선
- 명칭이나 내역과 같은 서술형 속성은 지양
- 속성의 수가 적은 것 우선
- 향후 변경 가능성이 적은 속성 우선

6.5 식별자 관계와 비식별자 관계

- **식별자 관계:** 부모 엔터티의 주식별자가 자식 엔터티의 주식별자에 포함되는 관계
 - 실선으로 표시
- **비식별자 관계:** 부모 엔터티의 주식별자가 자식 엔터티의 일반 속성으로 포함되는 관계
 - 점선으로 표시
- **관계 설정 기준:**
 - 자식 인스턴스가 부모 없이 독립적 존재 가능 → 비식별자 관계
 - 자식 인스턴스의 존재가 부모에 종속적 → 식별자 관계
 - 복합식별자가 너무 많아지면 → 비식별자 관계 고려

- 객체 재사용 필요시 → 비식별자 관계 고려
 - 이력 관리 엔터티 → 식별자 관계 고려
-

정리 및 핵심 포인트

1. 데이터 모델링은 현실 세계의 데이터를 추상화하여 표현하는 기법으로, 개념적/논리적/물리적 단계로 진행됩니다.
2. 데이터 모델의 세 가지 핵심 구성요소는 엔터티(실체), 속성(특성), 관계(연관성)입니다.
3. 관계형 데이터베이스는 SQL을 통해 데이터를 정의(DDL), 조작(DML), 제어(DCL)하며, 데이터 중복을 최소화합니다.
4. 엔터티는 업무에 필요한 정보 집합으로, 유형/개념/사건 엔터티로 나뉘고, 반드시 식별자와 속성을 포함해야 합니다.
5. 속성은 엔터티를 설명하는 최소 데이터 단위로, 기본/설계/파생 속성 및 PK/FK/일반 속성으로 구분됩니다.
6. 관계는 엔터티 간 연관성으로, 1:1, 1:M, N:M의 카디널리티와 필수/선택적 참여 특성을 갖습니다.
7. 식별자는 인스턴스를 구분하는 속성으로, 주/보조 식별자, 내부/외부 식별자, 단일/복합 식별자 등으로 분류됩니다.
8. 좋은 데이터 모델은 완전성, 중복 배제, 업무 규칙 반영, 데이터 재사용성, 명확한 의사소통, 통합성을 갖춰야 합니다.

1과목 데이터 모델링의 이해 - 2장 데이터 모델과 SQL

학습목표

- 정규화의 개념, 단계, 이상 현상을 이해하고 데이터 모델 품질을 향상시키는 방법을 학습하기
- 반정규화의 개념과 기법, 적용 시 고려사항을 파악하기
- 트랜잭션의 특성과 NULL의 특성을 정확히 이해하기
- 본질식별자와 인조식별자의 차이, 계층형 데이터 모델의 셀프 조인 등을 이해하기

목차

1. 정규화
 2. 반정규화
 3. 트랜잭션
 4. NULL
 5. 본질식별자 vs. 인조식별자
 6. 계층형 데이터 모델
-

1. 정규화

1.1 정규화의 정의

- **정규화(Normalization)**: 데이터의 일관성, 최소한의 데이터 중복, 최대한의 데이터 유연성을 확보하기 위한 데이터 분해 과정
- 엔티티를 상호 독립적인 관계로 분리하여 이상 현상(Anomaly)을 제거하는 것이 목적
- **함수적 종속성(Functional Dependency)**: 데이터 간의 종속 관계를 기반으로 정규화를 수행

1.2 이상 현상 (Anomaly)

- **삽입 이상(Insert Anomaly)**: 불필요한 데이터를 함께 삽입해야만 원하는 데이터를 추가할 수 있는 현상
- **갱신 이상(Update Anomaly)**: 중복 데이터 중 일부만 수정하여 데이터 불일치가 발생하는 현상
- **삭제 이상>Delete Anomaly**: 필요한 데이터까지 함께 삭제되는 현상

1.3 정규화 단계

제1정규형 (1NF)

- 모든 속성의 값이 **원자값(Atomic Value)**이어야 함
- 반복 그룹이나 다중값 속성을 제거
- 하나의 속성에는 하나의 값만 존재해야 함

제2정규형 (2NF)

- 1NF를 만족하면서 **부분 함수 종속(Partial Dependency)**을 제거
- 모든 비식별 속성이 주식별자 전체에 완전 함수 종속이어야 함
- 복합 식별자의 일부에만 종속되는 속성을 분리

제3정규형 (3NF)

- 2NF를 만족하면서 **이행 함수 종속(Transitive Dependency)**을 제거
- 비식별 속성 간의 종속 관계를 제거
- $A \rightarrow B \rightarrow C$ 에서 $A \rightarrow C$ 의 간접 종속이 존재하면 분리

BCNF (Boyce-Codd Normal Form)

- 3NF를 만족하면서 **결정자(Determinant)**가 모두 후보키이어야 함
- 후보키가 아닌 속성이 결정자인 경우를 제거
- 모든 결정자가 후보키인 상태

제4정규형 (4NF)

- **다치 종속(Multi-Valued Dependency)**을 제거
- 하나의 속성이 여러 개의 독립적인 다중값과 관련되는 경우를 분리

제5정규형 (5NF)

- **조인 종속(Join Dependency)**을 제거
- 분해된 테이블을 다시 조인했을 때 원래의 테이블과 동일해야 함

1.4 정규화와 성능

- 정규화는 **조회 성능을 저하시킬 수도, 향상시킬 수도 있음**
- 정규화를 하면 조인이 증가하여 조회 성능이 떨어질 수 있음
- 반면, 중복이 제거되어 데이터 일관성 유지 → 특정 조회 성능 향상
- **입력/수정/삭제 성능은 일반적으로 향상됨** (중복 데이터 감소로 처리량 감소)

2. 반정규화

2.1 반정규화의 정의

- **반정규화(Denormalization)**: 정규화된 엔터티, 속성, 관계에 대해 성능 향상과 개발 운영의 단순화를 위해 **중복, 통합, 분리** 등을 수행하는 데이터 모델링 기법
- 데이터 무결성이 깨질 수 있는 위험성이 있으므로 신중한 판단 필요

2.2 반정규화 적용 시 고려사항

- 반정규화 전에 먼저 고려할 사항:
 - 뷰(View) 활용으로 해결 가능한지 검토
 - 인덱스 조정으로 해결 가능한지 검토
 - 클러스터링 적용으로 해결 가능한지 검토
 - 파티셔닝이나 캐시 적용 등 다른 방안 검토
- 위 방안으로도 해결되지 않을 때 반정규화를 적용

2.3 반정규화 기법

테이블 반정규화

- **테이블 병합**: 1:1 관계 또는 1:M 관계의 테이블을 하나로 합침
- **테이블 분할**:
 - 수직 분할: 컬럼 단위로 테이블을 나눔 (자주 사용하는 컬럼과 그렇지 않은 컬럼을 분리)
 - 수평 분할: 행 단위로 테이블을 나눔 (지역별, 기간별 등으로 분리)
- **테이블 추가**:
 - 중복 테이블 추가: 다른 업무이거나 서버가 다른 경우 중복 테이블 생성
 - 통계 테이블 추가: 통계 데이터를 미리 계산하여 별도 테이블에 저장
 - 이력 테이블 추가: 변경 이력을 관리하기 위한 테이블 추가
 - 부분 테이블 추가: 자주 사용하는 컬럼만 모아 별도 테이블 생성

컬럼 반정규화

- **중복 컬럼 추가**: 조인 감소를 위해 다른 테이블의 컬럼을 중복 저장
- **파생 컬럼 추가**: 계산값을 미리 저장하여 조회 성능 향상 (예: 합계, 평균)
- **이력 테이블 컬럼 추가**: 최신 데이터를 빠르게 조회하기 위한 컬럼 추가
- **PK에 의한 컬럼 추가**: PK에 포함된 정보를 별도 컬럼으로 저장하여 인덱스 접근 편의 제공
- **응용시스템 오동작을 위한 컬럼 추가**: 이전 데이터와의 비교를 위한 컬럼

관계 반정규화

- **중복 관계 추가:** 조인 경로를 단축시키기 위해 이미 존재하는 관계를 중복으로 추가
-

3. 트랜잭션

3.1 트랜잭션의 정의

- **트랜잭션(Transaction):** 데이터베이스의 상태를 변환시키는 하나의 논리적 작업 단위
- 하나의 업무 단위로 묶여 있어, 전체가 수행되거나 전체가 취소되어야 함

3.2 트랜잭션의 특성 (ACID)

- **원자성(Atomicity):**
 - 트랜잭션의 연산은 전부 실행되거나 전부 취소되어야 함 (All or Nothing)
 - 일부만 수행되어서는 안 됨
- **일관성(Consistency):**
 - 트랜잭션 수행 전후로 데이터베이스의 상태가 일관되어야 함
 - 데이터 무결성 제약 조건을 항상 만족
- **고립성(Isolation):**
 - 하나의 트랜잭션이 실행 중일 때 다른 트랜잭션이 접근할 수 없음
 - 각 트랜잭션은 독립적으로 수행
- **영속성(Durability):**
 - 트랜잭션이 성공적으로 완료되면 그 결과는 영구적으로 반영
 - 시스템 장애가 발생하더라도 결과는 보존

3.3 트랜잭션과 데이터 모델링

- 하나의 트랜잭션에 관련된 데이터는 가능하면 하나의 엔터티에 배치
 - 트랜잭션의 단위가 데이터 모델의 엔터티 도출에 영향을 줌
-

4. NULL

4.1 NULL의 정의

- **NULL**: 아직 정의되지 않은 값, 값이 존재하지 않는 상태
- **NULL ≠ 공백(빈 문자열)**: NULL은 '값이 없음'이고, 공백은 '길이가 0인 문자열'
- **NULL ≠ 0**: NULL은 '값이 없음'이고, 0은 숫자 0이라는 값

4.2 NULL의 특성

- **산술 연산**: NULL이 포함된 산술 연산의 결과는 항상 NULL
 - 예: $100 + \text{NULL} = \text{NULL}$, $\text{NULL} \times 3 = \text{NULL}$
- **비교 연산**: NULL과의 비교 결과는 항상 Unknown(알 수 없음)
 - 예: $\text{NULL} = \text{NULL} \rightarrow \text{Unknown}$, $\text{NULL} > 0 \rightarrow \text{Unknown}$
- **논리 연산**: NULL은 3가지 값 논리(Three-Valued Logic)에 따름
 - $\text{TRUE AND Unknown} = \text{Unknown}$
 - $\text{FALSE AND Unknown} = \text{FALSE}$
 - $\text{TRUE OR Unknown} = \text{TRUE}$
 - $\text{FALSE OR Unknown} = \text{Unknown}$
 - $\text{NOT Unknown} = \text{Unknown}$

4.3 NULL 관련 함수

- **NVL(Oracle) / ISNULL(SQL Server)**: NULL을 지정한 값으로 치환
 - NVL(컬럼, 대체값): 컬럼이 NULL이면 대체값 반환
- **NULLIF**: 두 값이 같으면 NULL 반환, 다르면 첫 번째 값 반환
 - $\text{NULLIF}(\text{값1}, \text{값2})$: $\text{값1} = \text{값2}$ 이면 NULL, 아니면 값1
- **COALESCE**: 인수 중 첫 번째로 NULL이 아닌 값을 반환
 - $\text{COALESCE}(\text{값1}, \text{값2}, \text{값3}, \dots)$: 왼쪽부터 순서대로 NULL이 아닌 첫 번째 값 반환

4.4 집계 함수와 NULL

- **SUM, AVG, COUNT(컬럼), MAX, MIN**: NULL 값을 제외하고 연산
- **COUNT(*)**: NULL 포함 전체 행 수를 카운트
- **AVG 주의**: NULL을 제외한 행의 평균이므로, 전체 행의 평균과 다를 수 있음

5. 본질식별자 vs. 인조식별자

5.1 본질식별자

- **본질식별자(Natural Identifier)**: 업무에 의해 자연적으로 만들어진 식별자
- 업무적 의미를 가지고 있어 데이터의 의미를 직관적으로 파악 가능
- 예: 주민등록번호, 사업자등록번호, 수강신청(학번+과목번호)

5.2 인조식별자

- **인조식별자(Surrogate Identifier)**: 인위적으로 부여한 식별자
- 업무적 의미는 없으나 시스템 관리를 위해 생성
- 주로 자동 증가(Auto Increment) 번호나 시퀀스를 사용
- 예: 수강신청번호(시퀀스), 주문번호(자동채번)

5.3 인조식별자 사용 시 고려사항

- **불필요한 인조식별자 남용 주의**: 본질식별자만으로 충분한 경우 인조식별자 추가 지양
- **중복 데이터 발생 위험**: 인조식별자만으로는 업무적 중복을 방지하기 어려움
 - 예: 수강신청번호만 PK로 설정 시, 동일 학생이 같은 과목을 중복 신청 가능
 - 본질식별자를 유니크 제약조건으로 설정하거나 PK에 포함 필요
- **불필요한 인덱스 증가**: 인조식별자를 추가하면 관련 인덱스도 추가 필요
- **인조식별자가 적합한 경우**:
 - 본질식별자가 너무 많은 컬럼으로 구성된 복합식별자일 때
 - 자식 엔터티로 식별자가 계속 전이되어 관리가 복잡한 경우
 - 본질식별자의 값이 너무 길어 인덱스 성능에 영향을 줄 때

6. 계층형 데이터 모델

6.1 계층형 데이터 모델의 정의

- 엔터티 내에서 **자기 자신과의 관계(Self-Relationship)**를 가지는 데이터 모델
- 조직도, 카테고리, 메뉴 구조 등 트리(Tree) 구조의 데이터를 표현
- 하나의 테이블 내에서 상위-하위 관계를 부모키(FK)로 표현

6.2 셀프 조인 (Self Join)

- **셀프 조인**: 같은 테이블을 두 번 이상 참조하여 조인하는 방식
- 계층형 데이터에서 상위-하위 관계를 조회할 때 사용
- 테이블에 별칭(Alias)을 부여하여 마치 다른 테이블처럼 사용

- 예: 사원 테이블에서 사원과 해당 사원의 관리자 정보를 함께 조회

6.3 계층형 SQL

- Oracle의 계층형 쿼리:**
 - START WITH: 계층의 시작점(루트) 지정
 - CONNECT BY PRIOR: 부모-자식 관계의 방향 지정
 - LEVEL: 현재 계층의 깊이(레벨) 반환
 - ORDER SIBLINGS BY: 같은 레벨 내에서 정렬
- SQL Server의 계층형 쿼리:**
 - CTE(Common Table Expression)와 재귀 쿼리를 사용하여 구현
 - WITH RECURSIVE 구문 활용

6.4 계층형 데이터 모델 설계 시 고려사항

- 순환 참조(Cycle)가 발생하지 않도록 주의
- 루트 노드의 부모키는 NULL로 설정
- 계층의 깊이가 깊어지면 조회 성능 저하 가능
- 필요 시 경로(PATH) 컬럼을 추가하여 탐색 성능 향상

정리 및 핵심 포인트

- 정규화는 데이터 중복을 제거하고 이상 현상을 방지하기 위해 테이블을 분해하는 과정이며, 1NF(원자값) → 2NF(부분종속 제거) → 3NF(이행종속 제거) → BCNF(결정자=후보키) 순으로 진행됩니다.
- 반정규화는 성능 향상을 위해 정규화된 모델에 중복/통합/분리를 적용하는 기법이며, 반드시 다른 대안(뷰, 인덱스, 파티셔닝 등)을 먼저 검토한 후 적용해야 합니다.
- 트랜잭션은 ACID(원자성, 일관성, 고립성, 영속성) 특성을 가지며, 전부 수행되거나 전부 취소되는 논리적 작업 단위입니다.
- NULL은 '값이 없음'을 의미하며, 산술 연산 결과는 NULL, 비교 연산 결과는 Unknown이 됩니다. NVL, NULLIF, COALESCE 등의 함수로 처리합니다.
- 집계 함수(SUM, AVG, MAX, MIN, COUNT(컬럼))는 NULL을 제외하고 연산하며, COUNT(*)만 NULL을 포

함하여 전체 행 수를 계산합니다.

6. 본질식별자는 업무적으로 자연 발생한 식별자이고, 인조식별자는 시스템 편의를 위해 인위적으로 부여한 식별자입니다. 인조식별자 남용 시 중복 데이터 발생 위험이 있습니다.
7. 계층형 데이터 모델은 자기 자신과의 관계를 가지며, 셀프 조인이나 계층형 쿼리(START WITH/CONNECT BY, CTE)를 통해 조회합니다.
8. 정규화는 입력/수정/삭제 성능을 향상시키고, 반정규화는 조회 성능을 향상시키는 대신 데이터 무결성 위험이 있으므로 균형 있는 적용이 중요합니다.

2과목 SQL 기본과 활용 - 1장 SQL 기본

학습목표

- 관계형 데이터베이스에서 **SQL 표준 구문**(SELECT·WHERE·GROUP BY·HAVING ·ORDER BY)의 실행 순서를 이해하고 올바르게 작성할 수 있다.
- 단일행 함수·집계 함수·날짜/숫자/문자 함수**의 용도와 대표 예시를 설명하고 실습 쿼리를 작성할 수 있다.
- 조인 종류(INNER·OUTER·CROSS·NATURAL)**와 **JOIN** 조건을 구분·선택하여 두 개 이상 테이블을 효과적으로 결합할 수 있다.
- WHERE와 HAVING의 차이, 인덱스 활용, **동적 쿼리(WHERE 1=1 패턴)** 등 **실무 쿼리 작성 팁**을 적용해 성능과 유지보수성을 높일 수 있다.
- SQL을 사용한 **데이터 분석·무결성 검증·복잡한 보고서 생성** 사례를 설명하고, 다양한 비즈니스 요구에 맞춰 변형할 수 있다.

목차

1. SELECT 문
2. 함수
3. WHERE 절
4. GROUP BY 절, HAVING 절, ORDER BY 절
5. JOIN

1. SELECT 문

1.1 SELECT 문과 SELECT DISTINCT

SELECT 문의 기본 구조

```
SELECT [ALL/DISTINCT] 대상 컬럼명1, 대상 컬럼명2, ...  
FROM 대상 컬럼들이 있는 테이블명;
```

- ALL**: 기본 옵션으로, 중복된 데이터가 있어도 모두 출력 (생략 가능)
- DISTINCT**: 중복된 데이터가 있을 때 1건으로 처리하여 출력

- *: 애스터리스크(star)로, FROM 절의 테이블에 있는 모든 칼럼을 조회

예시

```
-- 특정 칼럼만 조회
SELECT EMPNO, ENAME, JOB FROM EMP;

-- 모든 칼럼 조회
SELECT * FROM EMP;

-- 중복 제거하여 조회 (JOB 칼럼의 고유 값만 출력)
SELECT DISTINCT JOB FROM EMP;
```

1.2 별칭(ALIAS) 사용하기

- 칼럼이나 테이블에 임시 이름을 지정하는 기능
- 칼럼 이름이 길거나 이해하기 어려울 때 쿼리 결과를 더 명확하게 하기 위해 사용

```
-- AS 키워드 사용
SELECT EMPNO AS 사원번호, ENAME AS 사원이름 FROM EMP;

-- 공백이 있는 별칭은 큰따옴표 사용 (오라클)
SELECT EMPNO AS "사원 번호", ENAME AS "사원 이름" FROM EMP;

-- SQL Server에서는 대괄호 사용
SELECT EMPNO AS [사원 번호], ENAME AS [사원 이름] FROM EMP;
```

1.3 산술 연산자와 문자 합성 연산자

산술 연산자

- +: 덧셈
- -: 뺄셈
- *: 곱셈
- /: 나눗셈
- %: 나머지 (오라클에서는 MOD 함수 사용)

```

-- 월급과 커미션 합산
SELECT ENAME, SAL, COMM, (SAL + NVL(COMM, 0)) AS TOTAL_COMP FROM EMP;

-- 월급에서 커미션 차감
SELECT ENAME, SAL, COMM, (SAL - NVL(COMM, 0)) AS NET_SAL FROM EMP;

-- 연봉 계산 (월급 * 12)
SELECT ENAME, SAL, (SAL * 12) AS ANNUAL_SAL FROM EMP;

-- 주급 계산 (월급 / 4)
SELECT ENAME, SAL, (SAL / 4) AS WEEKLY_SAL FROM EMP;

-- 짝수/홀수 판별 (오라클)
SELECT ENAME, EMPNO,
       CASE WHEN MOD(EMPNO, 2) = 0 THEN '짝수' ELSE '홀수' END AS ID_TYPE
FROM EMP;

```

문자 합성 연산자

- 오라클: || (두 개의 버티컬 라인)
- SQL Server: + 연산자

```

-- 오라클에서 문자열 합성
SELECT ENAME || ' is a ' || JOB AS EMPLOYEE_INFO FROM EMP;

-- SQL Server에서 문자열 합성
SELECT ENAME + ' is a ' + JOB AS EMPLOYEE_INFO FROM EMP;

-- CONCAT 함수 사용 (공통)
SELECT CONCAT(ENAME, CONCAT(' is a ', JOB)) AS EMPLOYEE_INFO FROM EMP;

```

2. 함수

2.1 SQL 내장 함수 개요

SQL 내장 함수는 데이터베이스에서 미리 정의된 함수로, 다음과 같이 분류됩니다:

- **단일행 함수:** 각 행에 대해 하나의 결과값을 반환
 - 문자형, 숫자형, 날짜형, 변환형 함수 등
- **다중행 함수:** 여러 행의 데이터를 집계하여 하나의 결과값을 반환
 - 집계 함수, 그룹 함수, 윈도우 함수 등

2.2 문자형 함수

함수	설명	예시
LOWER	문자열을 소문자로 변환	<code>SELECT LOWER('HELLO WORLD') → 'hello world'</code>
UPPER	문자열을 대문자로 변환	<code>SELECT UPPER('hello world') → 'HELLO WORLD'</code>
SUBSTR	문자열의 일부분을 추출	<code>SELECT SUBSTR('Hello World', 1, 5) → 'Hello'</code>
LENGTH	문자열의 길이를 반환	<code>SELECT LENGTH('Hello World') → 11</code>
TRIM	문자열의 양쪽 공백을 제거	<code>SELECT TRIM(' Hello World ') → 'Hello World'</code>
LTRIM	문자열의 왼쪽 공백을 제거	<code>SELECT LTRIM(' Hello World') → 'Hello World'</code>
RTRIM	문자열의 오른쪽 공백을 제거	<code>SELECT RTRIM('Hello World ') → 'Hello World'</code>
CONCAT	두 문자열을 연결	<code>SELECT CONCAT('Hello', ' World') → 'Hello World'</code>

2.3 숫자형 함수

함수	설명	예시
ABS	숫자의 절댓값을 반환	<code>SELECT ABS(-15) → 15</code>
SIGN	숫자의 부호를 반환	<code>SELECT SIGN(-10), SIGN(0), SIGN(10) → -1, 0, 1</code>
ROUND	숫자를 반올림	<code>SELECT ROUND(3.14159, 2) → 3.14</code>

TRUNC	숫자를 절삭	SELECT TRUNC(3.14159, 2) → 3.14
CEIL	숫자보다 크거나 같은 최소 정수	SELECT CEIL(4.3) → 5
FLOOR	숫자보다 작거나 같은 최대 정수	SELECT FLOOR(4.7) → 4
MOD	나눗셈 후 나머지	SELECT MOD(17, 5) → 2
POWER	거듭제곱	SELECT POWER(2, 3) → 8
SQRT	제곱근	SELECT SQRT(9) → 3

2.4 날짜형 함수

함수	설명	예시
SYSDATE	현재 날짜와 시각 반환	SELECT SYSDATE FROM DUAL
ADD_MONTHS	지정한 개월 수 만큼 날짜를 더함	SELECT ADD_MONTHS(SYSDATE, 3)
MONTHS_BETWEEN	두 날짜 사이의 개월 수를 계산	SELECT MONTHS_BETWEEN(SYSDATE, '2024-01-01')
NEXT_DAY	지정한 날짜 이후의 특정 요일 날짜	SELECT NEXT_DAY('2024-07-20', '월요일')
LAST_DAY	해당 월의 마지막 날짜	SELECT LAST_DAY(SYSDATE)
EXTRACT	날짜에서 특정 부분 추출	SELECT EXTRACT(YEAR FROM SYSDATE)

2.5 변환형 함수

함수	설명	예시
TO_CHAR	날짜/숫자를 문자열로 변환	SELECT TO_CHAR(SYSDATE, 'YYYY-MM-DD')

TO_DATE	문자열을 날짜로 변환	SELECT TO_DATE('2024-07-21', 'YYYY-MM-DD')
TO_NUMBER	문자열을 숫자로 변환	SELECT TO_NUMBER('12345')
CAST	데이터 타입 변환	SELECT CAST('12345' AS NUMBER)

2.6 NULL 관련 함수

함수	설명	예시
NVL (오라클)	NULL인 경우 지정한 값으로 대체	SELECT ENAME, NVL(COMM, 0) FROM EMP
COALESCE	최초의 NULL이 아닌 값 반환	SELECT COALESCE(COMM, SAL, 0) FROM EMP
NULLIF	두 값이 같으면 NULL, 다르면 첫 번째 값	SELECT NULLIF(COMM, 0) FROM EMP

2.7 CASE 문

조건에 따라 다른 값을 반환하는 조건문

```
-- 간단 CASE 문
SELECT ENAME, JOB,
       CASE JOB
         WHEN 'CLERK' THEN 'Clerk Job'
         WHEN 'SALESMAN' THEN 'Sales Job'
         WHEN 'MANAGER' THEN 'Manager Job'
         ELSE 'Other Job'
       END AS JOB_DESCRIPTION
FROM EMP;

-- 검색 CASE 문
SELECT ENAME, SAL,
       CASE
         WHEN SAL < 1000 THEN 'Low Salary'
```

```
WHEN SAL BETWEEN 1000 AND 3000 THEN 'Medium Salary'
WHEN SAL > 3000 THEN 'High Salary'
ELSE 'Unknown'
END AS SALARY_GRADE
FROM EMP;
```

3. WHERE 절

3.1 WHERE 절의 개념

- SQL 쿼리에서 특정 조건을 만족하는 행만 선택하기 위해 사용
- 데이터베이스에서 원하는 데이터를 필터링하는 데 필수적
- FROM 절 다음에 위치

```
SELECT * FROM EMP WHERE SAL > 2000 AND DEPTNO = 10;
```

3.2 연산자 종류와 사용법

비교 연산자

- =: 같다
- <>, !=, ^=: 같지 않다
- >, <, >=, <=: 크다, 작다, 크거나 같다, 작거나 같다

```
-- 급여가 3000인 직원
SELECT ENAME, JOB, SAL FROM EMP WHERE SAL = 3000;

-- 급여가 3000이 아닌 직원
SELECT ENAME, JOB, SAL FROM EMP WHERE SAL <> 3000;

-- 급여가 2000 이상인 직원
SELECT ENAME, JOB, SAL FROM EMP WHERE SAL >= 2000;
```

SQL 연산자

- **BETWEEN a AND b**: a와 b 사이의 값
- **IN (list)**: 목록에 있는 값 중 하나와 일치
- **LIKE '패턴'**: 패턴과 일치하는 문자열
- **IS NULL**: NULL 값
- **NOT BETWEEN, NOT IN, IS NOT NULL**: 위 조건들의 부정형

```
-- 급여가 1500에서 3000 사이인 직원
SELECT ENAME, JOB, SAL FROM EMP WHERE SAL BETWEEN 1500 AND 3000;

-- 부서 번호가 10, 20, 30인 직원
SELECT ENAME, JOB, DEPTNO FROM EMP WHERE DEPTNO IN (10, 20, 30);

-- 이름이 'S'로 시작하는 직원
SELECT ENAME FROM EMP WHERE ENAME LIKE 'S%';

-- 이름에 'R'이 포함된 직원
SELECT ENAME FROM EMP WHERE ENAME LIKE '%R%';

-- 이름의 두 번째 글자가 'A'인 직원
SELECT ENAME FROM EMP WHERE ENAME LIKE '_A%';

-- 커미션이 NULL인 직원
SELECT ENAME, COMM FROM EMP WHERE COMM IS NULL;
```

논리 연산자

- **AND**: 모든 조건이 참
- **OR**: 하나 이상의 조건이 참
- **NOT**: 조건의 부정

```
-- 급여가 2000 이상이고 매니저인 직원
SELECT ENAME, SAL, JOB FROM EMP WHERE SAL >= 2000 AND JOB = 'MANAGER';

-- 급여가 2000 초과이거나 부서가 20인 직원
SELECT ENAME, SAL, DEPTNO FROM EMP WHERE SAL > 2000 OR DEPTNO = 20;

-- 급여가 1000 이하가 아니거나 매니저인 직원
SELECT ENAME, SAL, JOB FROM EMP WHERE NOT SAL <= 1000 OR JOB = 'MANAGER';
```

3.3 연산자 우선순위와 괄호 사용

연산자 우선순위:

1. 괄호 ()
2. 산술 연산자
3. 문자열 연산자
4. 비교/SQL 연산자
5. NOT
6. AND
7. OR

```
-- 괄호를 사용한 복잡한 조건
SELECT ENAME, SAL, DEPTNO FROM EMP
WHERE (SAL > 2000 AND DEPTNO = 10) OR (SAL BETWEEN 1500 AND 3000);
```

3.4 이스케이프 문자

LIKE 검색에서 %, _ 등의 특수문자를 값으로 검색할 때 사용

```
-- 파일 이름에 '%2024'가 포함된 파일 검색
SELECT FILE_NAME FROM FILES WHERE FILE_NAME LIKE 'report\%2024%' ESCAPE '\';
```

3.5 실무 팁: WHERE 1=1 사용

동적 쿼리 생성 시 조건절을 쉽게 추가/제거할 수 있게 하는 기법

```
SELECT * FROM EMP
WHERE 1=1
AND SAL > 2000
AND DEPTNO = 10;
-- 조건 추가/제거가 용이함
```

4. GROUP BY 절, HAVING 절, ORDER BY 절

4.1 집계 함수 개요

집계 함수는 여러 행의 데이터를 하나의 결과값으로 집계하는 데 사용되는 함수입니다.

- **SUM**: 합계
- **AVG**: 평균
- **COUNT**: 개수
- **MAX**: 최댓값
- **MIN**: 최솟값

```
-- 전체 급여 합계
SELECT SUM(SAL) FROM EMP;

-- 평균 급여
SELECT AVG(SAL) FROM EMP;

-- 직원 수
SELECT COUNT(*) FROM EMP;

-- 최고 급여
SELECT MAX(SAL) FROM EMP;

-- 최저 급여
SELECT MIN(SAL) FROM EMP;
```

4.2 GROUP BY 절

데이터를 그룹화하여 집계 함수를 각 그룹에 적용할 수 있게 합니다.

```
-- 부서별 급여 합계와 평균
SELECT DEPTNO, SUM(SAL) AS TOTAL_SAL, AVG(SAL) AS AVG_SAL
FROM EMP
GROUP BY DEPTNO;

-- 부서와 직무별 급여 합계
SELECT DEPTNO, JOB, SUM(SAL) AS TOTAL_SAL
FROM EMP
GROUP BY DEPTNO, JOB
ORDER BY DEPTNO;
```

주의사항:

- GROUP BY 절에서는 ALIAS를 사용할 수 없습니다.
- SELECT 절에 GROUP BY 절에 없는 칼럼은 사용할 수 없습니다(집계 함수 제외).

4.3 HAVING 절

GROUP BY로 그룹화된 결과에 조건을 적용할 때 사용합니다.

```
-- 평균 급여가 2000을 초과하는 부서 조회
SELECT DEPTNO, AVG(SAL) AS AVG_SAL
FROM EMP
GROUP BY DEPTNO
HAVING AVG(SAL) > 2000;

-- 직원이 3명 이상인 직무 조회
SELECT JOB, COUNT(*) AS EMP_COUNT
FROM EMP
GROUP BY JOB
HAVING COUNT(*) >= 3;
```

WHERE 절과 HAVING 절의 차이:

- **WHERE:** 그룹화 전 개별 행을 필터링 (집계 함수 사용 불가)
- **HAVING:** 그룹화 후 그룹을 필터링 (집계 함수 사용 가능)

```
-- WHERE와 HAVING 절 함께 사용
```

```
SELECT DEPTNO, AVG(SAL) AS AVG_SAL
FROM EMP
WHERE SAL >= 1000 -- 개별 급여 1000 이상인 직원만 대상
GROUP BY DEPTNO
HAVING AVG(SAL) > 2000; -- 그 중 평균 급여 2000 초과 부서만 선택
```

4.4 ORDER BY 절

쿼리 결과를 특정 칼럼 기준으로 정렬할 때 사용합니다.

```
-- 오름차순 정렬 (ASC 생략 가능)
SELECT ENAME, SAL
FROM EMP
ORDER BY SAL ASC;

-- 내림차순 정렬
SELECT ENAME, SAL
FROM EMP
ORDER BY SAL DESC;

-- 여러 열로 정렬 (부서번호 오름차순, 동일 부서 내 급여 내림차순)
SELECT ENAME, JOB, SAL, DEPTNO
FROM EMP
ORDER BY DEPTNO ASC, SAL DESC;
```

SQL 실행 순서:

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY

5. JOIN

5.1 JOIN의 개념

JOIN은 두 개 이상의 테이블에서 관련된 데이터를 결합하여 새로운 결과 집합을 생성하는 과정입니다. 조인의 기본 목적은 테이블 간의 관계를 활용하여 필요한 정보를 효율적으로 검색하는 것입니다.

5.2 JOIN의 기본 문법

```
SELECT 별칭1.칼럼명, 별칭2.칼럼명
FROM 테이블1 AS 별칭1
JOIN 테이블2 AS 별칭2
ON 별칭1.칼럼명 = 별칭2.칼럼명;
```

5.3 JOIN의 종류

INNER JOIN

두 테이블에서 공통된 값을 가진 행만 반환합니다.

```
SELECT E.EMPNO, E.ENAME, D.DNAME
FROM EMP E INNER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO;
```

LEFT OUTER JOIN

왼쪽 테이블의 모든 행과 오른쪽 테이블의 일치하는 행을 반환합니다. 일치하는 행이 없으면 NULL로 표시됩니다.

```
SELECT E.EMPNO, E.ENAME, D.DNAME
FROM EMP E LEFT OUTER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO;

-- 오라클 문법
SELECT E.EMPNO, E.ENAME, D.DNAME
FROM EMP E, DEPT D
WHERE E.DEPTNO = D.DEPTNO(+);
```

RIGHT OUTER JOIN

오른쪽 테이블의 모든 행과 왼쪽 테이블의 일치하는 행을 반환합니다.

```
SELECT E.EMPNO, E.ENAME, D.DNAME
FROM EMP E RIGHT OUTER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO;
```

FULL OUTER JOIN

양쪽 테이블의 모든 행을 반환합니다. 일치하지 않는 행은 NULL로 표시됩니다.

```
SELECT E.EMPNO, E.ENAME, D.DNAME
FROM EMP E FULL OUTER JOIN DEPT D
ON E.DEPTNO = D.DEPTNO;
```

CROSS JOIN

두 테이블 간의 모든 행을 조합합니다(카티시안 곱).

```
SELECT E.ENAME, D.DNAME
FROM EMP E CROSS JOIN DEPT D;
```

5.4 등가 JOIN과 비등가 JOIN

등가 JOIN (EQUI JOIN)

= 연산자를 사용하여 두 테이블의 칼럼이 같은 값인 경우를 조인합니다.

```
SELECT E.ENAME, D.DNAME
FROM EMP E JOIN DEPT D
ON E.DEPTNO = D.DEPTNO;
```

비등가 JOIN (NON-EQUI JOIN)

= 외의 연산자(>, <, BETWEEN 등)를 사용하여 조인합니다.

```
SELECT E.ENAME, E.SAL, S.GRADE
FROM EMP E JOIN SALGRADE S
ON E.SAL BETWEEN S.LOSAL AND S.HISAL;
```

5.5 NATURAL JOIN과 USING 조건절

NATURAL JOIN

두 테이블에 동일한 이름을 가진 모든 칼럼에 대해 자동으로 조인합니다.

```
SELECT ENAME, DNAME
FROM EMP NATURAL JOIN DEPT;
```

USING 조건절

조인할 특정 칼럼을 명시합니다. 동일한 이름의 칼럼이 하나 이상 있을 때 유용합니다.

```
SELECT ENAME, DNAME, DEPTNO
FROM EMP JOIN DEPT USING (DEPTNO);
```

5.6 3개 이상 테이블 JOIN

```
SELECT E.ENAME, D.DNAME, L.CITY
FROM EMP E
JOIN DEPT D ON E.DEPTNO = D.DEPTNO
JOIN LOCATIONS L ON D.LOC_ID = L.LOC_ID;
```

5.7 실무 주의사항

- JOIN 시 카디널리티(유일성)가 낮은 칼럼으로 조인하면 결과 행 수가 급격히 증가할 수 있습니다.
 - 적절한 인덱스가 있는 칼럼으로 JOIN해야 성능이 좋습니다.
 - 대용량 테이블 JOIN 시 실행 계획을 확인하고 최적화해야 합니다.
-

정리 및 핵심 포인트

1. **SELECT** 문은 “FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY” 순서로 실행된다—순서를 기억해야 복잡한 쿼리에서 오류를 피할 수 있다.
2. 함수는 **단일행/다중행**으로 구분된다. 단일행 함수는 각 행별 결과, 집계 함수는 그룹 전체 결과를 반환한다.
3. WHERE 절은 **그룹화 이전** 필터, HAVING 절은 **그룹화 이후** 필터이다—집계 함수를 쓰려면 HAVING을 사용한다.
4. **INNER JOIN**은 교집합, **OUTER JOIN**은 포함 관계를 유지하며 NULL 로 채운다. 조인 조건에 인덱스가 잡히도록 설계하면 성능을 크게 개선할 수 있다.
5. 동적 쿼리를 자주 작성한다면 **WHERE 1=1** 패턴으로 조건을 손쉽게 추가/제거하고, **별칭·CASE·서브쿼리**를 적절히 사용해 가독성과 재사용성을 높인다.
6. SQL은 단순 질의어를 넘어 **데이터 통합·보고·품질 관리** 등 거의 모든 데이터 작업의 공통 언어다. 핵심 문법과 패턴을 체득하면 현업 데이터베이스뿐 아니라 BI·ETL·데이터 분석 도구에서도 즉시 활용할 수 있다.

2과목 SQL 기본과 활용 - 2장 SQL 활용

학습목표

- 서브쿼리(단일·다중·연관)와 **인라인 뷰**를 적재적소에 사용하여 복잡한 조건을 단순화하고, 뷰를 활용해 보안을 강화하거나 재사용성을 높일 수 있다.
- **집합 연산자** (UNION / UNION ALL / INTERSECT / MINUS · EXCEPT)의 차이를 이해하고, 데이터 통합·차이 분석 시 적절히 선택할 수 있다.
- **ROLLUP·CUBE·GROUPING SETS**를 이용한 다차원 집계와 **GROUPING, CASE/DECODE**로 읽기 쉬운 레포트를 작성할 수 있다.
- **윈도우 함수**(ROW_NUMBER, RANK, SUM OVER 등)를 PARTITION BY·ORDER BY·FRAME 절과 함께 사용해 누적·순위·이전/다음 행 분석을 구현할 수 있다.
- **Top N 및 페이징** 쿼리를 Oracle(ROWNUM)·SQL Server(TOP)·표준 SQL(FETCH FIRST) 방식으로 작성하고 성능 이슈를 피할 수 있다.
- **셀프 조인과 계층형 질의(START WITH ... CONNECT BY)** 구문으로 트라·조직도·BOM 등 계층 데이터를 효과적으로 조회할 수 있다.
- **PIVOT/UNPIVOT**으로 행·열을 자유롭게 전환해 교차 테이블을 만들고, **정규표현식** 함수·연산자로 문자열 패턴을 검증·추출·변환할 수 있다.

목차

1. 서브쿼리

- 서브쿼리 개요
- 서브쿼리 종류
- 서브쿼리 활용법
- 뷰의 개념과 활용법

2. 집합 연산자

- 집합 연산자 개념
- 집합 연산자 종류
- 집합 연산자 활용 예시

3. 그룹 함수

- 그룹 함수 개념
- 다차원 집계
- GROUPING/CASE/DECODE 활용

4. 윈도우 함수

- 윈도우 함수 개념
- OVER 절 활용법
- 윈도우 함수 종류

5. Top N 쿼리

- ROWNUM 슈도 칼럼
- Top N 쿼리 활용법

6. 셀프 조인과 계층형 질의

- 셀프 조인 개념 및 활용
- 계층형 질의 개념 및 문법

7. PIVOT 절과 UNPIVOT 절

- PIVOT 절 개념 및 문법
- UNPIVOT 절 개념 및 문법

8. 정규표현식

- 정규표현식 개념
- 정규표현식 연산자
- 정규표현식 함수

서브쿼리

서브쿼리 개요

서브쿼리의 기초 개념

쿼리(Query)는 하나의 명령 문장을 의미하며, 이를 메인쿼리(Mainquery)라고 합니다. 그 명령 문장 안에 또 다른 쿼리가 있는데, 이를 서브쿼리(Subquery) 또는 부질의라고 합니다.

서브쿼리는 메인쿼리 내부에 괄호로 감싸진 형태로 존재하며, 다양한 절(WHERE, FROM, SELECT 등)에서 사용될 수 있습니다.

서브쿼리 사용 시점

- 복잡한 조건 필터링
- 계산된 칼럼 생성
- 임시 테이블(INLINE-VIEW) 활용
- 조인보다 간단한 존재 여부 판정(예: EXISTS, IN)

서브쿼리 종류

동작 방식에 따른 분류

1. 연관 서브쿼리(Correlated Subquery)

- 메인 쿼리의 각 행과 관련되어 처리
- 메인 쿼리 한 행을 읽을 때마다 서브 쿼리가 실행됨

```
SELECT ENAME, DEPTNO, SAL
FROM EMP E1
WHERE SAL > (SELECT AVG(SAL)
              FROM EMP E2
              WHERE E1.DEPTNO = E2.DEPTNO);
```

2. 비연관 서브쿼리(Un-Correlated Subquery)

- 메인 쿼리와 독립적으로 실행
- 한 번만 실행되어 결과를 메인 쿼리에 전달

```
SELECT ENAME, SAL
FROM EMP
WHERE SAL > (SELECT AVG(SAL) FROM EMP);
```

반환 형태에 따른 분류

1. 단일행 서브쿼리(Single Row Subquery)

- 하나의 행만 반환
- 단일행 비교 연산자(=, >, <, >=, <=, <>) 사용

```
SELECT ENAME, SAL
FROM EMP
WHERE SAL > (SELECT AVG(SAL) FROM EMP);
```

2. 다중행 서브쿼리(Multiple Row Subquery)

- 여러 행을 반환
- 다중행 비교 연산자(IN, ANY, ALL, EXISTS) 사용

```
SELECT ENAME, SAL
FROM EMP
WHERE DEPTNO IN (SELECT DEPTNO FROM DEPT WHERE LOC = 'NEW YORK');
```

3. 다중칼럼 서브쿼리(Multiple Column Subquery)

- 여러 칼럼을 반환
- 복합 조건 비교에 사용

```
SELECT *
FROM EMP
WHERE (JOB, DEPTNO) IN (SELECT JOB, DEPTNO FROM EMP WHERE ENAME = 'SMITH');
```

서브쿼리 활용법

WHERE 절에서의 서브쿼리

```
SELECT ENAME, SAL
FROM EMP
WHERE SAL > (SELECT AVG(SAL) FROM EMP)
ORDER BY SAL DESC;
```

HAVING 절에서의 서브쿼리

```
SELECT DEPTNO, AVG(SAL)
FROM EMP
GROUP BY DEPTNO
HAVING AVG(SAL) > (SELECT AVG(SAL) FROM EMP);
```

SELECT 절에서의 서브쿼리

```
SELECT ENAME, SAL,
       (SELECT AVG(SAL) FROM EMP) AS AVG_SAL
FROM EMP
ORDER BY SAL DESC;
```

FROM 절에서의 서브쿼리(인라인 뷰)

```
SELECT E.ENAME, E.SAL, A.AVG_SAL
FROM EMP E,
     (SELECT AVG(SAL) AS AVG_SAL FROM EMP) A
WHERE E.SAL > A.AVG_SAL
ORDER BY SAL DESC;
```

DML에서의 서브쿼리 활용

1. INSERT와 서브쿼리

```
INSERT INTO EMP (EMPNO, ENAME, SAL)
VALUES ((SELECT MAX(EMPNO) + 1 FROM EMP), 'NEW_EMP', 3000);
```

2. UPDATE와 서브쿼리

```
UPDATE EMP
SET SAL = SAL * 1.1
WHERE DEPTNO = (SELECT DEPTNO FROM DEPT WHERE DNAME = 'SALES');
```

3. DELETE와 서브쿼리

```
DELETE FROM EMP
WHERE SAL < (SELECT AVG(SAL) FROM EMP);
```

뷰(VIEW)의 개념과 활용법

뷰의 개념

뷰는 하나 이상의 테이블에 있는 데이터의 부분집합을 논리적으로 표현한 가상 테이블입니다. 실제 데이터를 가지고 있지 않고, 데이터를 추출하는 SQL문을 저장하고 있습니다.

뷰의 장점

- 복잡한 쿼리 단순화
- 데이터 보안 강화(필요한 칼럼만 제공)
- 데이터 독립성 확보
- 동일 데이터에 대한 다양한 관점 제공

뷰의 생성과 활용

```
CREATE VIEW EMP_DEPT_V AS
SELECT E.EMPNO, E.ENAME, D.DNAME, E.SAL
FROM EMP E, DEPT D
WHERE E.DEPTNO = D.DEPTNO;

-- 뷰 사용
SELECT * FROM EMP_DEPT_V WHERE SAL > 2000;
```

집합 연산자

집합 연산자 개념

집합 연산자는 두 개 이상의 SELECT 문의 결과를 하나의 결과로 결합하는 연산자입니다. 특히 회계나 통계 분석 등에서 유용하게 사용됩니다.

집합 연산자 종류

UNION (합집합, 중복 제거)

두 개의 SELECT 문 결과를 하나로 합치고, 중복된 행은 제거합니다.

```
SELECT DEPTNO, DNAME FROM DEPT
UNION
SELECT DEPTNO, JOB FROM EMP;
```

UNION ALL (합집합, 중복 포함)

두 개의 SELECT 문 결과를 하나로 합치고, 중복된 행도 그대로 포함합니다.

```
SELECT DEPTNO FROM DEPT
UNION ALL
SELECT DEPTNO FROM EMP;
```

INTERSECT (교집합)

두 SELECT 문의 결과에 모두 포함된 행만 반환합니다.

```
SELECT DEPTNO FROM DEPT
INTERSECT
SELECT DEPTNO FROM EMP;
```

MINUS / EXCEPT (차집합)

첫 번째 SELECT 문의 결과에서 두 번째 SELECT 문의 결과에 포함된 행을 제외한 나머지를 반환합니다.
(Oracle은 MINUS, SQL Server는 EXCEPT 사용)

```
SELECT DEPTNO FROM DEPT  
MINUS  
SELECT DEPTNO FROM EMP;
```

집합 연산자 사용 조건

- 각 SELECT 문의 칼럼 수가 동일해야 합니다.
- 대응되는 칼럼의 데이터 타입이 같거나 호환 가능해야 합니다.
- 결과는 첫 번째 SELECT 문의 칼럼 이름을 따릅니다.
- ORDER BY 절은 전체 결과에 대해 마지막에 한 번만 사용합니다.

그룹 함수

그룹 함수 개념

그룹 함수는 데이터를 다차원적으로 집계하는 기능을 제공합니다. GROUP BY 절과 함께 사용하여 그룹별 합계, 소계, 총계 등의 다양한 통계 정보를 산출할 수 있습니다.

다차원 집계

ROLLUP

ROLLUP은 GROUP BY 절과 함께 사용하여 지정된 칼럼의 소계와 총계를 자동으로 산출합니다. 계층적 구조로 집계를 생성합니다.

```
SELECT DEPTNO, JOB, SUM(SAL) AS TOTAL_SALARY  
FROM EMP
```

```
GROUP BY ROLLUP(DEPTNO, JOB)
ORDER BY DEPTNO;
```

이 쿼리는 다음 그룹별 총합을 계산합니다:

- 각 부서와 직무 조합
- 각 부서별
- 전체 총합

CUBE

CUBE는 ROLLUP과 유사하지만, 지정된 칼럼들의 가능한 모든 조합에 대해 집계를 수행합니다.

```
SELECT DEPTNO, JOB, SUM(SAL)
FROM EMP
GROUP BY CUBE(DEPTNO, JOB);
```

이 쿼리는 다음 그룹별 총합을 계산합니다:

- 각 부서와 직무 조합
- 각 부서별
- 각 직무별
- 전체 총합

GROUPING SETS

GROUPING SETS는 원하는 특정 그룹의 집계만 수행할 수 있도록 해줍니다. ROLLUP이나 CUBE의 일부 기능만 필요할 때 유용합니다.

```
SELECT DEPTNO, JOB, SUM(SAL)
FROM EMP
GROUP BY GROUPING SETS((DEPTNO, JOB), (DEPTNO), (JOB), ());
```

이 쿼리는 다음 그룹별 총합을 계산합니다:

- 각 부서와 직무 조합

- 각 부서별
- 각 직무별
- 전체 총합

GROUPING/CASE/DECODE 활용

GROUPING 함수

GROUPING 함수는 특정 칼럼이 집계에 사용되었는지 여부를 반환합니다. 집계에 사용되었으면 0, 그렇지 않으면 1을 반환합니다.

```
SELECT
    CASE GROUPING(DEPTNO) WHEN 1 THEN '전체 부서' ELSE TO_CHAR(DEPTNO) END AS
DEPTNO,
    CASE GROUPING(JOB) WHEN 1 THEN '전체 직무' ELSE JOB END AS JOB,
    SUM(SAL) AS TOTAL_SALARY
FROM EMP
GROUP BY ROLLUP(DEPTNO, JOB);
```

CASE와 DECODE 활용

CASE 표현식이나 DECODE 함수를 사용하여 집계 결과의 가독성을 높일 수 있습니다.

```
-- Oracle DECODE 예시
SELECT
    DECODE(GROUPING(DEPTNO), 1, '전체 부서', TO_CHAR(DEPTNO)) AS DEPTNO,
    DECODE(GROUPING(JOB), 1, '전체 직무', JOB) AS JOB,
    SUM(SAL) AS TOTAL_SALARY
FROM EMP
GROUP BY ROLLUP(DEPTNO, JOB);

-- CASE 문 예시
SELECT
    CASE
        WHEN GROUPING(DEPTNO) = 1 THEN '전체 부서'
        ELSE TO_CHAR(DEPTNO)
    END AS DEPTNO,
```

```

CASE
    WHEN GROUPING(JOB) = 1 THEN '전체 직무'
    ELSE JOB
END AS JOB,
SUM(SAL) AS TOTAL_SALARY
FROM EMP
GROUP BY ROLLUP(DEPTNO, JOB);

```

윈도우 함수

윈도우 함수 개념

윈도우 함수는 행과 행 간의 관계를 쉽게 정의하기 위해 제공되는 함수입니다. 특정 윈도우(파티션)에 대해 계산을 수행합니다. 집계 함수와 달리 행이 그대로 유지되면서 값이 추가됩니다.

윈도우 함수 기본 구문

```

윈도우함수(인자) OVER (
    [PARTITION BY 칼럼]
    [ORDER BY 칼럼]
    [ROWS | RANGE BETWEEN 시작점 AND 끝점]
)

```

- **PARTITION BY:** 계산을 적용할 행 그룹 지정
- **ORDER BY:** 파티션 내에서 행 순서 지정
- **ROWS | RANGE:** 프레임 지정(현재 행을 기준으로 계산할 범위)

윈도우 함수 종류

순위 관련 함수

1. **ROW_NUMBER()**
 - 파티션 내에서 순차적인 번호 부여(중복 값에도 순차적인 번호)

```
SELECT ENAME, SAL,  
       ROW_NUMBER() OVER(ORDER BY SAL DESC) AS RN  
FROM EMP;
```

2. RANK()

- 파티션 내에서 순위를 매기되, 동일한 값은 동일한 순위(다음 순위는 건너뛴)

```
SELECT ENAME, SAL,  
       RANK() OVER(ORDER BY SAL DESC) AS RANK_SAL  
FROM EMP;
```

3. DENSE_RANK()

- RANK와 유사하나, 동일한 순위 다음에 순차적인 값을 부여(건너뛰지 않음)

```
SELECT ENAME, SAL,  
       DENSE_RANK() OVER(ORDER BY SAL DESC) AS DENSE_RANK_SAL  
FROM EMP;
```

집계 관련 함수

1. SUM() OVER

- 파티션별 합계 계산

```
SELECT ENAME, DEPTNO, SAL,  
       SUM(SAL) OVER(PARTITION BY DEPTNO) AS DEPT_SUM  
FROM EMP;
```

2. AVG() OVER

- 파티션별 평균 계산

```
SELECT ENAME, DEPTNO, SAL,
       AVG(SAL) OVER(PARTITION BY DEPTNO) AS DEPT_AVG
FROM EMP;
```

3. MAX() OVER, MIN() OVER

- 파티션별 최댓값, 최솟값 계산

```
SELECT ENAME, DEPTNO, SAL,
       MAX(SAL) OVER(PARTITION BY DEPTNO) AS DEPT_MAX,
       MIN(SAL) OVER(PARTITION BY DEPTNO) AS DEPT_MIN
FROM EMP;
```

행 이동 관련 함수

1. LAG()

- 이전 행의 값을 참조

```
SELECT ENAME, HIREDATE, SAL,
       LAG(SAL) OVER(ORDER BY HIREDATE) AS PREV_SAL
FROM EMP;
```

2. LEAD()

- 다음 행의 값을 참조

```
SELECT ENAME, HIREDATE, SAL,
       LEAD(SAL) OVER(ORDER BY HIREDATE) AS NEXT_SAL
FROM EMP;
```

누적 관련 함수

1. CUME_DIST()

- 누적 분포 값을 계산(현재 행보다 작거나 같은 값의 누적 개수 / 전체 행 수)

```
SELECT ENAME, SAL,  
       CUME_DIST() OVER(ORDER BY SAL) AS CUM_DIST  
FROM EMP;
```

2. PERCENT_RANK()

- 백분위 순위 계산((현재 행 순위 - 1) / (전체 행 수 - 1))

```
SELECT ENAME, SAL,  
       PERCENT_RANK() OVER(ORDER BY SAL) AS PCT_RANK  
FROM EMP;
```

Top N 쿼리

ROWNUM 슈도 칼럼

ROWNUM은 Oracle에서 제공하는 가상 칼럼(pseudo column)으로 쿼리 결과의 각 행에 1부터 시작하는 순차적인 번호를 부여합니다.

```
SELECT ROWNUM, ENAME, SAL  
FROM EMP  
WHERE ROWNUM <= 5;
```

Top N 쿼리 활용법

Oracle에서의 Top N 쿼리(ROWNUM 활용)

```
-- 잘못된 방법 (원하는 결과가 나오지 않음)  
SELECT ROWNUM, ENAME, SAL
```

```

FROM EMP
WHERE ROWNUM <= 5
ORDER BY SAL DESC;

-- 올바른 방법 (인라인 뷰 활용)
SELECT ROWNUM, E.*
FROM (SELECT ENAME, SAL
      FROM EMP
      ORDER BY SAL DESC) E
WHERE ROWNUM <= 5;

```

SQL Server에서의 Top N 쿼리

```

-- SQL Server의 TOP 사용
SELECT TOP 5 ENAME, SAL
FROM EMP
ORDER BY SAL DESC;

```

표준 SQL의 FETCH 절 사용

```

-- 표준 SQL의 FETCH 문 사용(Oracle 12c 이상, SQL Server 2012 이상)
SELECT ENAME, SAL
FROM EMP
ORDER BY SAL DESC
FETCH FIRST 5 ROWS ONLY;

```

페이징 처리

```

-- 11~20번째 행 조회
SELECT *
FROM (
    SELECT ROWNUM AS RN, E.*
    FROM (SELECT * FROM EMP ORDER BY SAL DESC) E
    WHERE ROWNUM <= 20
)

```

```
WHERE RN >= 11;
```

셀프 조인과 계층형 질의

셀프 조인 개념 및 활용

셀프 조인(Self Join)은 하나의 테이블을 마치 두 개의 테이블인 것처럼 조인하는 기법입니다. 주로 계층적 관계나 순차적 관계를 조회할 때 사용됩니다.

```
-- 사원과 관리자 정보 함께 조회
SELECT E.EMPNO, E.ENAME AS EMP_NAME,
       M.EMPNO AS MGR_EMPNO, M.ENAME AS MGR_NAME
FROM EMP E, EMP M
WHERE E.MGR = M.EMPNO;

-- 또는 JOIN 문법 사용
SELECT E.EMPNO, E.ENAME AS EMP_NAME,
       M.EMPNO AS MGR_EMPNO, M.ENAME AS MGR_NAME
FROM EMP E LEFT JOIN EMP M
ON E.MGR = M.EMPNO;
```

계층형 질의 개념 및 문법

계층형 질의는 테이블에 저장된 계층적 구조의 데이터를 조회하는 방법입니다. Oracle에서는 START WITH와 CONNECT BY 절을 사용합니다.

```
-- 기본 구문
SELECT [컬럼 목록], LEVEL
FROM [테이블]
START WITH [최상위 조건]
CONNECT BY [PRIOR 부모키 = 자식키 | 자식키 = PRIOR 부모키];
```

계층형 질의 주요 요소

- **START WITH:** 계층 구조의 시작점(루트 노드) 지정
- **CONNECT BY:** 부모-자식 관계를 정의
- **LEVEL:** 계층의 깊이를 나타내는 의사 컬럼
- **CONNECT_BY_ROOT:** 현재 행의 최상위 조상 노드를 반환
- **SYS_CONNECT_BY_PATH:** 루트부터 현재 노드까지의 경로를 문자열로 반환
- **CONNECT_BY_ISLEAF:** 현재 행이 리프 노드인지 여부(1 또는 0)

계층형 질의 예제

```
-- 조직도 조회(최상위 관리자부터)
SELECT ENAME, EMPNO, MGR, LEVEL,
       LPAD(' ', (LEVEL-1)*2) || ENAME AS ORG_CHART
FROM EMP
START WITH MGR IS NULL
CONNECT BY PRIOR EMPNO = MGR
ORDER SIBLINGS BY ENAME;
```

PIVOT 절과 UNPIVOT 절

PIVOT 절 개념 및 문법

PIVOT은 행을 열로 변환해주는 기능을 수행합니다. 특히 보고서 형태의 출력이나 교차 테이블을 생성할 때 유용합니다.

```
-- 기본 구문
SELECT *
FROM (기준 테이블)
PIVOT (
    집계함수(집계대상컬럼)
    FOR 피벗컬럼 IN (피벗값1, 피벗값2, ...)
);
```

PIVOT 활용 예제

```
-- 부서별, 직무별 사원 수 피벗 테이블
SELECT *
FROM (SELECT DEPTNO, JOB FROM EMP)
PIVOT (
    COUNT(*)
    FOR JOB IN ('CLERK' AS CLERK, 'MANAGER' AS MANAGER, 'ANALYST' AS ANALYST,
    'SALESMAN' AS SALESMAN)
)
ORDER BY DEPTNO;
```

UNPIVOT 절 개념 및 문법

UNPIVOT은 PIVOT의 반대 기능으로, 열을 행으로 변환합니다.

```
-- 기본 구문
SELECT *
FROM (피벗된 테이블)
UNPIVOT (
    값_저장할_컬럼이름
    FOR 컬럼_저장할_컬럼이름 IN (컬럼1, 컬럼2, ...)
);
```

UNPIVOT 활용 예제

```
-- 피벗된 테이블을 다시 원래 형태로 변환
SELECT *
FROM (피벗된 테이블 또는 서브쿼리)
UNPIVOT (
    EMP_COUNT
    FOR JOB_TYPE IN (CLERK, MANAGER, ANALYST, SALESMAN)
);
```

정규표현식

정규표현식 개념

정규표현식은 문자열 내에서 특정 패턴을 검색, 추출, 변환하는 데 사용되는 강력한 도구입니다.

정규표현식 연산자

연산자	설명	예시
.	임의의 단일 문자	'a.c'는 'abc', 'adc' 등과 일치
	OR 연산자	'a b'는 'a' 또는 'b'와 일치
*	앞 요소가 0회 이상 반복	'ab*'는 'a', 'ab', 'abb' 등과 일치
+	앞 요소가 1회 이상 반복	'ab+'는 'ab', 'abb' 등과 일치(단, 'a'는 미포함)
?	앞 요소가 0회 또는 1회 반복	'ab?'는 'a', 'ab'와 일치
{n}	앞 요소가 정확히 n회 반복	'ab{3}'는 'abbb'와 일치

| {n,} | 앞 요소가 n회 이상 반복

정리 및 핵심 포인트

1. **서브쿼리 위치(SELECT·FROM·WHERE·HAVING)마다 역할이 다르다** — 인라인 뷰는 복잡한 조인 대신 임시 테이블처럼 활용하면 가독성이 크게 향상된다.
2. 집합 연산자는 **칼럼 수·데이터형이 일치해야** 하며, **UNION ALL이 UNION보다 빠르지만 중복을 제거하지 않는다**. 마지막 ORDER BY는 전체 결과에 한 번만 작성한다.
3. **ROLLUP은 계층적 합계, CUBE는 모든 조합 합계, GROUPING SETS는 선택적 합계를 만든다**. GROUPING 함수로 ‘총계’ 표시를 제어하면 보고서 품질이 높아진다.
4. 윈도우 함수는 **행을 지우지 않고 통계열을 추가한다**. ORDER BY만 있고 PARTITION BY가 없으면 전 행을 한 그룹으로 본다—프레임 범위도 성능에 영향.
5. **Top N 쿼리에서 ROWNUM은 정렬 후에 매겨야 정확하다**. 표준 SQL : FETCH FIRST n ROWS ONLY를 기억해 두면 DBMS 이식성을 확보할 수 있다.
6. 셀프 조인은 **자체 테이블의 부모·자식 관계나 시간대별 이전 상태 비교에 필수적**이다. 계층형 질의에서는 LEVEL, SYS_CONNECT_BY_PATH 등 의사 컬럼이 유용하다.
7. **PIVOT/UNPIVOT은 집계 함수가 필수**이며, 열 이름은 IN 절에 미리 명시하는 것이 안전하다. 정규표현식은 오

라클의 REGEXP_LIKE·REPLACE·SUBSTR 등과 함께 쓰면 데이터 정제 속도가 획기적으로 빨라진다.

2과목 SQL 기본과 활용 - 3장 관리구분

학습목표

- **SQL 명령 분류(DML·TCL·DDL·DCL)**를 구분하고, 각 유형이 데이터·트랜잭션·스키마·권한에 미치는 역할을 설명할 수 있다.
- INSERT·UPDATE·DELETE·MERGE 등 **DML** 문을 사용해 데이터를 삽입·수정·삭제·동기화할 수 있다.
- COMMIT·ROLLBACK·SAVEPOINT를 포함한 **TCL** 명령으로 트랜잭션을 제어하고, ACID 특성을 검증·적용할 수 있다.
- CREATE·ALTER·DROP·TRUNCATE 등 **DDL** 명령어로 테이블·제약조건·인덱스 등을 정의·변경·삭제할 수 있다.
- **DCL**(GRANT·REVOKE·ROLE)을 이용해 사용자·권한을 효율적으로 관리하고, 보안 정책을 수립할 수 있다.

목차

1. **DML** (Data Manipulation Language, 데이터 조작어)
2. **TCL** (Transaction Control Language, 트랜잭션 제어어)
3. **DDL** (Data Definition Language, 데이터 정의어)
4. **DCL** (Data Control Language, 데이터 제어어)

1. DML (데이터 조작어)

DML은 데이터베이스의 테이블에 데이터를 추가, 수정, 삭제하는 명령어입니다. 주요 명령어로는 INSERT, UPDATE, DELETE, MERGE가 있습니다.

1.1 INSERT 문

INSERT 문은 데이터베이스 테이블에 새로운 데이터를 추가할 때 사용하는 SQL 명령어입니다.

1.1.1 단일행 INSERT 문

기본 형식

```
INSERT INTO 테이블명 (칼럼1, 칼럼2, ...)
```

```
VALUES (값1, 값2, ...);
```

모든 칼럼에 값을 삽입할 경우 칼럼 이름을 생략할 수 있습니다.

```
INSERT INTO 테이블명  
VALUES (값1, 값2, ...);
```

예시

```
-- 특정 칼럼만 지정하여 삽입  
INSERT INTO EMP (EMPNO, ENAME, JOB, HIREDATE)  
VALUES (101, 'YONGMOON', 'MANAGER', '2023-08-01');  
  
-- 모든 칼럼 값 지정하여 삽입  
INSERT INTO EMP  
VALUES (102, 'YONGMOON2', 'CLERK', '7902', '2023-08-03', 2000, NULL, 20);
```

1.1.2 서브쿼리를 이용한 다중행 INSERT 문

다른 테이블에서 데이터를 추출해 새로운 테이블에 삽입할 때 유용합니다.

예시

```
-- EMP_MNG 테이블 생성  
CREATE TABLE EMP_MNG (  
    EMPNO NUMBER(4,0),  
    ENAME VARCHAR2(10),  
    JOB VARCHAR2(9),  
    MGR NUMBER(4,0),  
    HIREDATE DATE,  
    SAL NUMBER(7,2),  
    COMM NUMBER(7,2),  
    DEPTNO NUMBER(2,0)  
);  
  
-- 서브쿼리를 이용한 다중행 INSERT  
INSERT INTO EMP_MNG (  
    EMPNO,
```

```

        ENAME,
        JOB,
        HIREDATE,
        DEPTNO
    )
    SELECT
        EMPNO,
        ENAME,
        JOB,
        HIREDATE,
        DEPTNO
    FROM
        EMP
    WHERE
        DEPTNO = 20;

```

서브쿼리를 이용하면 특정 조건에 맞는 다수의 데이터를 한 번에 다른 테이블로 삽입할 수 있습니다. 위 예시에서는 부서 번호가 20인 사원 정보만 선택적으로 삽입했습니다.

1.2 UPDATE 문

UPDATE 문은 테이블에 이미 존재하는 데이터를 수정할 때 사용합니다.

기본 형식

```

UPDATE 테이블명
SET 칼럼1 = 값1, 칼럼2 = 값2, ...
WHERE 조건;

```

예시

```

-- 여러 칼럼을 동시에 UPDATE
UPDATE EMP
SET SAL = 200, COMM = 300
WHERE ENAME = 'YONGMOON';

-- 서브쿼리를 이용한 UPDATE
UPDATE EMP
SET SAL = (SELECT AVG(SAL) FROM EMP WHERE DEPTNO = 10)

```

```
WHERE ENAME = 'YONGMOON';
```

두 번째 예시는 서브쿼리를 사용하여 DEPTNO가 10인 부서의 평균 급여를 계산하고, 그 값을 사용하여 YONGMOON의 SAL 값을 업데이트합니다.

1.3 DELETE 문

DELETE 문은 테이블의 특정 행을 삭제할 때 사용합니다.

기본 형식

```
DELETE FROM 테이블명
WHERE 조건;
```

예시

```
DELETE FROM EMP
WHERE ENAME = 'YONGMOON2';
```

DELETE와 TRUNCATE의 차이점

특성	DELETE	TRUNCATE
작동 방식	행 단위로 데이터를 삭제	테이블 구조를 유지한 채 모든 데이터를 한 번에 제거
속도	각 행을 개별적으로 삭제하므로 상대적으로 느림	전체 테이블을 한 번에 비우므로 일반적으로 더 빠름
트랜잭션 로그	각 삭제 작업을 트랜잭션 로그에 기록	최소한의 트랜잭션 로그만 생성
롤백 가능성	트랜잭션 내에서 롤백이 가능	일반적으로 롤백이 불가능
WHERE 절 사용	WHERE 절을 사용하여 특정 조건의 데이터만 삭제할 수 있음	WHERE 절을 사용할 수 없으며, 항상 전체 테이블을 비움
인덱스 영향	인덱스를 유지	인덱스를 재설정

1.4 MERGE 문

MERGE 문은 데이터의 삽입, 업데이트, 삭제 작업을 하나의 명령문으로 처리할 수 있도록 도와주는 명령어입니다. 주로 소스 테이블의 데이터를 타겟 테이블에 반영할 때 사용됩니다.

기본 형식

```
MERGE INTO 타겟_테이블 타겟(별칭)
USING 소스_테이블 소스(별칭)
ON (타겟.조건_칼럼 = 소스.조건_칼럼)
WHEN MATCHED THEN
    UPDATE SET 타겟.칼럼1 = 소스.칼럼1, ...
WHEN NOT MATCHED THEN
    INSERT (칼럼1, 칼럼2, ...)
VALUES (소스.칼럼1, 소스.칼럼2, ...);
```

예시

```
MERGE INTO EMP_MNG M
USING EMP E
ON (M.EMPNO = E.EMPNO)
WHEN MATCHED THEN
    UPDATE SET
        M.SAL = E.SAL,
        M.COMM = E.SAL
WHEN NOT MATCHED THEN
    INSERT (EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO)
VALUES (E.EMPNO, E.ENAME, E.JOB, E.MGR, E.HIREDATE, E.SAL, E.COMM,
        E.DEPTNO);
```

이 예시에서는 EMP_MNG와 EMP 테이블에서 EMPNO가 일치하는 행이 있을 경우, EMP 테이블의 SAL 값을 EMP_MNG 테이블의 SAL과 COMM 칼럼에 UPDATE합니다. 일치하지 않는 EMP 테이블의 행들은 EMP_MNG 테이블에 새로운 행으로 INSERT됩니다.

2. TCL (트랜잭션 제어어)

TCL은 여러 DML 명령어로 구성된 트랜잭션을 관리하는 명령어입니다. 주요 명령어로는 COMMIT, ROLLBACK, SAVEPOINT가 있습니다.

2.1 트랜잭션의 기본 개념

트랜잭션이란 데이터베이스 내에서 실행되는 하나의 논리적 작업 단위를 말하며, 여러 개의 연산을 포함할 수 있습니다.

트랜잭션의 ACID 속성

- **원자성(Atomicity)**: 모든 연산이 성공적으로 완료되거나 아무것도 실행되지 않은 상태를 유지
- **일관성(Consistency)**: 트랜잭션은 데이터베이스의 일관성을 유지
- **고립성(Isolation)**: 다른 트랜잭션의 영향을 받지 않고 독립적으로 실행
- **지속성(Durability)**: 성공적으로 수행된 트랜잭션의 결과는 시스템 오류가 발생해도 유지

2.2 COMMIT

COMMIT을 실행하면 트랜잭션이 종료되고, 해당 트랜잭션에서 이루어진 모든 변경 사항이 데이터베이스에 영구적으로 저장됩니다.

예시

```
-- 1. 직원 삭제
DELETE FROM EMP WHERE ENAME = 'YONGMOON' OR ENAME = 'YONGMOON2';

-- 2. 새로운 직원 추가
INSERT INTO EMP (EMPNO, ENAME, JOB, MGR, HIREDATE, SAL, COMM, DEPTNO)
VALUES (7935, 'JAMESBOND', 'ANALYST', 7566, TO_DATE('2023-08-19', 'YYYY-MM-DD'), 3500, NULL, 20);

-- 3. 급여 10% 인상
UPDATE EMP SET SAL = SAL * 1.1 WHERE EMPNO = 7935;

-- 4. 부서 번호 변경
UPDATE EMP SET DEPTNO = 30 WHERE EMPNO = 7902;

-- 5. 모든 변경 사항 확정
COMMIT;
```

COMMIT 이후에는 해당 트랜잭션에 대한 ROLLBACK이 불가능합니다.

2.3 ROLLBACK

ROLLBACK은 트랜잭션의 모든 변경 사항을 취소하고, 트랜잭션 시작 시점으로 되돌리는 명령어입니다.

예시

```
-- 1. JAMESBOND(7935) 급여 40% 인상
UPDATE EMP SET SAL = SAL * 1.4 WHERE EMPNO = 7935;

-- 2. JAMESBOND 부서 번호를 30으로 수정
UPDATE EMP SET DEPTNO = 30 WHERE EMPNO = 7935;

-- 3. 변경사항 전부 취소
ROLLBACK;
```

ROLLBACK 실행 후에는 트랜잭션 시작 전 상태로 되돌아갑니다.

2.4 SAVEPOINT

SAVEPOINT는 트랜잭션 내에서 특정 지점을 지정하고, 후에 해당 지점까지만 롤백할 수 있게 하는 명령어입니다.

예시

```
-- 트랜잭션 시작
BEGIN;

-- 1. 첫 번째 INSERT
INSERT INTO EMP (EMPNO, ENAME, JOB, SAL, DEPTNO)
VALUES (8001, 'AAAAA', 'CLERK', 1500, 10);

-- 2. SAVEPOINT sp1
SAVEPOINT sp1;

-- 3. 두 번째 INSERT
INSERT INTO EMP (EMPNO, ENAME, JOB, SAL, DEPTNO)
VALUES (8002, 'BBBBB', 'MANAGER', 2500, 20);
```

```
-- 4. SAVEPOINT sp2
SAVEPOINT sp2;

-- 5. 세 번째 INSERT
INSERT INTO EMP (EMPNO, ENAME, JOB, SAL, DEPTNO)
VALUES (8003, 'CCCCC', 'ANALYST', 3000, 30);

-- 6. 오류로 sp2로 롤백
ROLLBACK TO sp2;

-- 7. 나머지 작업 확정
COMMIT;
```

위 예시에서 세 번째 INSERT는 취소되지만, 첫 번째와 두 번째 INSERT는 커밋됩니다. COMMIT 후에는 SAVEPOINT가 모두 무효화됩니다.

3. DDL (데이터 정의어)

DDL은 데이터베이스에서 테이블, 인덱스, 뷰, 스키마 등 데이터베이스 객체를 정의하고 관리하기 위한 SQL 명령어들의 집합입니다.

3.1 CREATE TABLE

CREATE TABLE 명령은 새로운 테이블을 생성하는 SQL 명령어입니다.

기본 형식

```
CREATE TABLE 테이블명 (
  칼럼1 데이터타입 [제약조건],
  칼럼2 데이터타입 [제약조건],
  ...
);
```

예시

```
CREATE TABLE STUDENT (
```

```
STUDENT_ID NUMBER(5) PRIMARY KEY,
NAME VARCHAR2(50) NOT NULL,
AGE NUMBER(3),
ENROLL_DATE DATE DEFAULT SYSDATE,
DEPT_ID NUMBER(3) REFERENCES DEPARTMENT(DEPT_ID)
);
```

참조 무결성 옵션

외래 키를 사용할 때 참조 무결성을 유지하기 위한 옵션을 지정할 수 있습니다:

- **ON DELETE CASCADE:** 부모 테이블의 행이 삭제되면 자식 테이블의 참조하는 행도 함께 삭제
- **ON DELETE SET NULL:** 부모 테이블의 행이 삭제되면 자식 테이블의 참조하는 열을 NULL로 설정
- **ON DELETE SET DEFAULT:** 부모 테이블의 행이 삭제되면 자식 테이블의 참조하는 열을 기본값으로 설정
- **ON DELETE NO ACTION:** 부모 테이블의 행을 삭제하려고 할 때 자식 테이블에 참조하는 행이 있으면 삭제 불가
- **ON DELETE RESTRICT:** NO ACTION과 유사하게 동작

예시

```
CREATE TABLE CHILD_TABLE_CASCADE (
  CHILD_ID NUMBER PRIMARY KEY,
  PARENT_ID NUMBER,
  CONSTRAINT FK_PARENT_CASCADE FOREIGN KEY (PARENT_ID)
  REFERENCES PARENT_TABLE (PARENT_ID) ON DELETE CASCADE
);
```

3.2 ALTER TABLE

ALTER TABLE 명령어는 기존 테이블의 구조를 수정할 때 사용하는 SQL 명령어입니다.

주요 기능

- **칼럼 추가:** ALTER TABLE 테이블명 ADD 칼럼명 데이터타입 [제약조건]
- **칼럼 삭제:** ALTER TABLE 테이블명 DROP COLUMN 칼럼명
- **칼럼명 변경:** ALTER TABLE 테이블명 RENAME COLUMN 기존칼럼명 TO 새칼럼명

- 데이터 타입 변경: ALTER TABLE 테이블명 MODIFY 컬럼명 새데이터타입
- 제약 조건 추가: ALTER TABLE 테이블명 ADD CONSTRAINT 제약조건명 제약조건타입 (컬럼명)
- 제약 조건 삭제: ALTER TABLE 테이블명 DROP CONSTRAINT 제약조건명
- 기본값 설정: ALTER TABLE 테이블명 MODIFY 컬럼명 데이터타입 DEFAULT 기본값

예시

```
-- 컬럼 추가
ALTER TABLE EMPLOYEE ADD PHONE_NUMBER VARCHAR2(15) NOT NULL;

-- 컬럼 삭제
ALTER TABLE EMPLOYEE DROP COLUMN PHONE_NUMBER;

-- 컬럼명 변경
ALTER TABLE EMPLOYEE RENAME COLUMN NAME TO FULL_NAME;

-- 데이터 타입 변경
ALTER TABLE EMPLOYEE MODIFY SALARY NUMBER(10, 2);

-- 제약 조건 추가
ALTER TABLE EMPLOYEE ADD CONSTRAINT CHK_SALARY CHECK (SALARY > 0);

-- 제약 조건 삭제
ALTER TABLE EMPLOYEE DROP CONSTRAINT CHK_SALARY;

-- 기본값 설정
ALTER TABLE EMPLOYEE MODIFY JOIN_DATE DATE DEFAULT SYSDATE;
```

3.3 RENAME TABLE

테이블 이름을 변경하는 명령어입니다.

오라클 예시

```
RENAME EMP TO EMP_BACKUP;
```

SQL Server 예시

```
sp_rename 'EMP', 'EMP_BACKUP';
```

3.4 DROP TABLE

DROP TABLE 명령어는 테이블을 삭제하는 명령어로, 테이블의 모든 데이터와 구조를 함께 제거합니다.

```
DROP TABLE EMP_BACKUP;
```

3.5 TRUNCATE TABLE

TRUNCATE TABLE은 테이블의 모든 데이터를 빠르게 삭제하나 테이블 구조는 유지합니다.

```
TRUNCATE TABLE EMP_BACKUP;
```

DROP TABLE과 TRUNCATE TABLE의 차이점

특성	DROP TABLE	TRUNCATE TABLE
구조 삭제	테이블 구조와 데이터 모두 삭제	데이터만 삭제, 구조 유지
복구 가능성	일반적으로 복구 불가능	일반적으로 복구 불가능
관련 객체	인덱스, 트리거, 제약 조건 등 모두 삭제	인덱스, 트리거, 제약 조건은 유지
공간 할당	테이블 공간 반환	테이블 공간 초기화

4. DCL (데이터 제어어)

DCL은 데이터베이스에 대한 접근 권한을 관리하는 명령어입니다. 주요 명령어로는 GRANT와 REVOKE가 있습니다.

4.1 사용자 계정과 권한

데이터베이스 시스템에서는 사용자 계정을 생성하고 해당 계정에 필요한 권한을 부여합니다.

사용자 생성 예시 (Oracle)

```
CREATE USER username IDENTIFIED BY password;
```

4.2 권한의 종류

- **시스템 권한:** 데이터베이스 전체에 대한 권한 (예: CREATE TABLE, CREATE VIEW)
- **객체 권한:** 특정 객체에 대한 권한 (예: SELECT, INSERT, UPDATE, DELETE)

4.3 GRANT

GRANT 명령어는 사용자에게 권한을 부여합니다.

객체 권한 부여 형식

```
GRANT 권한1, 권한2, ... ON 객체명 TO 사용자명 [WITH GRANT OPTION];
```

WITH GRANT OPTION: 부여받은 권한을 다른 사용자에게 부여할 수 있는 권한까지 부여

예시

```
-- SELECT와 INSERT 권한 부여
GRANT SELECT, INSERT ON EMP TO bbigter;

-- 권한 전달 옵션 포함
GRANT SELECT ON EMP TO bbigter WITH GRANT OPTION;
```

4.4 REVOKE

REVOKE 명령어는 사용자에게 부여한 권한을 회수합니다.

형식

```
REVOKE 권한1, 권한2, ... ON 객체명 FROM 사용자명;
```

예시

```
REVOKE SELECT, INSERT ON EMP FROM bbiger;
```

4.5 ROLE을 이용한 권한 관리

ROLE은 여러 권한을 묶어 그룹화한 것으로, 권한 관리를 효율적으로 할 수 있습니다.

ROLE 생성 및 권한 부여

```
-- ROLE 생성
CREATE ROLE hr_role;

-- ROLE에 권한 부여
GRANT SELECT, INSERT, UPDATE ON EMP TO hr_role;
GRANT SELECT ON DEPT TO hr_role;

-- 사용자에게 ROLE 부여
GRANT hr_role TO user1, user2;
```

ROLE 삭제

```
DROP ROLE hr_role;
```

ROLE을 사용하면 여러 사용자에게 동일한 권한을 부여하고 관리하는 작업을 간소화할 수 있습니다.

정리 및 핵심 포인트

1. **DML**은 행 단위 데이터 조작에 쓰이며, MERGE로 삽입·갱신을 한 번에 처리할 수 있다—대량 작업 시 서브쿼리·버퍼 캐싱을 활용하면 성능이 향상된다.
2. **TCL**은 트랜잭션의 일관성을 보장한다. COMMIT은 변경사항을 영구화하고, ROLLBACK은 트랜잭션 시작점

또는 SAVEPOINT까지 복원한다. ACID 원칙을 기억하라.

3. **DDL**은 객체 구조를 다룬다. TRUNCATE는 DELETE보다 빠르지만 롤백이 불가능하며, DROP은 구조·데이터·연관 객체를 모두 제거한다.
4. **DCL**은 접근 제어의 핵심이다. GRANT로 권한을 부여하고, REVOKE로 회수하며, ROLE로 여러 권한을 묶어 다수 사용자에게 일괄 적용한다.
5. 실무에서는 **권한 최소화 원칙(Principle of Least Privilege)**을 적용해 보안 사고를 예방하고, 트랜잭션 관리 규칙을 문서화해 데이터 무결성을 유지한다.